SHIELDMCP: An Interactive System for Runtime Security of Model Context Protocol Tool Calls

Saurabh Yergattikar
Independent Researcher
Dublin, CA, USA
saurabh.ssy@gmail.com

Abstract

The Model Context Protocol (MCP) lets large language model agents call external tools, and that tool channel has become a security liability: poisoned tool metadata, injected instructions in tool responses, and malicious servers can subvert an agent while never touching the user's prompt. These risks are increasingly well documented, but hard to *experience*—most defenses exist as prose or prototypes. We present SHIELDMCP, an interactive, deployment-ready system that makes both the attack and the runtime defense tangible in a browser. SHIELDMCP operates as a transparent proxy between the LLM client and any MCP server, performing three-stage validation—*pre-call* description integrity, *outbound* parameter sanitization, and *post-response* analysis—without modifying either the agent or the server. The demonstration exposes the full pipeline through a browser interface in which visitors launch curated attacks from five threat families, watch each validation stage execute live with per-token highlighting of the detected payload, and submit their own tool descriptions, call parameters, or tool responses to a playground that runs the real detector. On a benchmark of 487 attack and 200 benign scenarios across 40 MCP servers, SHIELDMCP reduces the attack success rate from 70.9% (undefended) to 8.6% with rule-based detectors and to near-zero with optional fine-tuned classifiers, while preserving 95% of benign task utility and adding under 20 ms of median latency per tool call. SHIELDMCP is open-source under the Apache-2.0 license and installable with a single `pip` command.

1 Introduction

The security of the Model Context Protocol (MCP; Salih et al., 2025) tool layer is, today, mostly discussed on paper. A growing literature catalogues how tool descriptions can be poisoned (Wang et al., 2025), how tool responses carry indirect prompt injections (Greshake et al., 2023; An et al., 2025), and how malicious servers slip through registries (Song et al., 2025; Zhao et al., 2025). Yet a practitioner deciding whether these risks are real for *their* deployment, or whether a proposed defense actually blocks them, has little they can run: most defenses are described in prose or shipped as research prototypes, not as something an engineer installs in front of a live agent and watches work.

This demonstration closes that gap. SHIELDMCP is a runtime security proxy for the MCP tool layer, packaged so that anyone can experience the threat and the defense in a browser within seconds of `pip install`. Visitors launch real attacks from five families against real MCP servers, watch the proxy inspect each tool call stage by stage, see the offending payload highlighted token by token as it is blocked, and then paste their own inputs into a playground that runs the same detector. The detection framework and threat taxonomy that SHIELDMCP operationalizes are developed in a companion research paper; the contribution *here* is a deployable, inspectable system that turns that analysis into something people can touch.

Concretely, the demonstration contributes: (i) a transparent, drop-in MCP security proxy that needs no changes to the agent or server and speaks both the stdio and SSE transports, so it wraps existing deployments with a single command; (ii) a browser interface that executes the *real* three-stage pipeline live—every verdict is computed on the spot, and each validation stage is animated with in-place highlighting of the tokens that triggered it; and (iii) an interactive playground plus a one-command reproducible benchmark (487 attacks, 200 benign tasks, 40 servers) that lets an audience stress-test the detector and re-run every number in this paper.

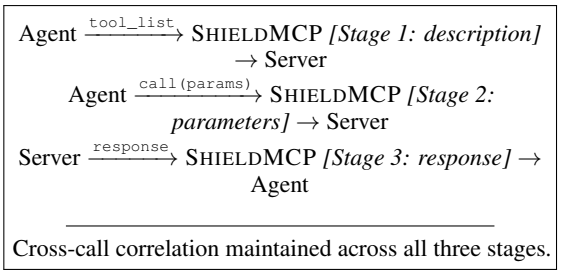

Figure 1: SHIELDMCP interposes on the JSON-RPC stream as a transparent proxy, validating tool descriptions before planning, outbound parameters before they leave the client, and inbound responses before the agent reads them.

2 Scope and Audience

The demonstration covers five attack families—tool poisoning, indirect prompt injection through responses, supply-chain compromise, rug pulls, and cross-tool chains—each represented by a runnable scenario in the interface. We do not re-derive the taxonomy here (it is the subject of the companion paper); rather, each family is present so that a visitor can trigger it and observe the corresponding defense. What unifies them, and what makes the live demonstration instructive, is that all five ride on the *tool channel* rather than the user prompt, so they are invisible to conventional input/output guardrails—a point that is far more convincing to watch than to read.

The demonstration is built for three audiences. **Engineers** evaluating MCP security can install SHIELDMCP and see, on their own machine, whether a given tool or server trips the detector before trusting it in production. **Researchers** in agent security get an open, one-command benchmark and a playground for probing and comparing detectors. **Educators and practitioners** get a concrete, interactive way to teach why the tool layer is a distinct attack surface—the demo was designed to run fully offline on a laptop at a poster session, with no API keys and no network.

3 System Architecture

SHIELDMCP operates as a transparent proxy on the JSON-RPC transport between the LLM client and connected MCP servers (Figure 1). Because it speaks the MCP wire protocol, it is a drop-in interposition: agents send standard requests and servers receive standard requests. A single command wraps any stdio server,

```
shieldmcp proxy - python server.py
```

and an SSE proxy mode covers HTTP-based servers. The pipeline performs three stages.

Stage 1: Description integrity (pre-call). Before the agent's planning phase, SHIELDMCP inspects every registered tool description with two checks. A *structural* analyzer flags description text that no legitimate tool needs: hidden Unicode (zero-width joiners, right-to-left overrides), markup-based concealment, and imperative patterns buried in metadata fields. A *semantic* check then scores how strongly the description reads as a covert instruction to the agent rather than a neutral capability description. SHIELDMCP also maintains a baseline registry of vetted tool signatures (name, description hash, parameter schema) and raises a high-severity alert when a previously seen tool changes—the rug-pull detector—requiring explicit re-authorization.

Stage 2: Parameter sanitization (outbound). When the agent constructs a call, SHIELDMCP validates the outbound parameters against the tool's declared schema (type, range) and scans string arguments for payloads that a downstream system could interpret: SQL injection fragments, shell command injection, path traversal, and prompt-injection strings targeting other LLMs in the chain.

Stage 3: Response analysis (post-response). Tool responses are the primary vector for indirect prompt injection, so this is the most consequential stage. SHIELDMCP performs (i) *instruction detection*, identifying segments of the response that contain directive language using either pattern rules or a token-level classifier; (ii) *context-boundary enforcement*, wrapping responses in explicit delimiters that the agent is conditioned to treat as untrusted data—a lightweight, prompt-engineering approximation of the control/data separation advocated by Debenedetti et al. (2025); and (iii) *cross-call correlation*, maintaining a session-level dependency graph so that a response which triggers an unexpected follow-on tool call raises a high-severity chaining alert.

Pluggable detection backends. Stages 1 and 3 each expose a backend switch. The default `heuristic` backend is dependency-light and requires no training or GPU, making it the conservative operating point for deployment. An optional

`classifier` backend loads fine-tuned DistilBERT models—a binary sequence classifier for descriptions and a token classifier for responses—for higher recall. Backends are selected in a YAML config or at the command line, so operators trade recall against footprint without code changes.

4 The Interactive Demonstration

The demonstration is a single-page web application, served by the same Python package as the proxy and launched with `shieldmcp demo`. Every verdict shown in the browser is computed live by the real pipeline—nothing is mocked or pre-recorded. The interface (Figure 2) has three regions.

Attack gallery. A left panel lists curated scenarios spanning all five threat families plus a benign control. Selecting one streams the corresponding MCP traffic through the proxy and animates the three stage cards in sequence: each card transitions from *scanning* to a *pass*/*warn*/*block* verdict, reports its measured latency and alert count, and—critically for pedagogy—renders the offending tool description, parameter, or response with the exact substrings that triggered detection highlighted in place. When a stage blocks, the pipeline halts there (mirroring production behavior), and a verdict banner summarizes the end-to-end outcome and latency.

Live benchmark. A right panel surfaces the aggregate evaluation—overall attack success rate, benign pass rate, and a per-family attack-success bar chart—read directly from the benchmark result file, so the numbers a visitor sees on screen are the same numbers reported in Section 5.

Playground. A modal lets visitors paste an arbitrary tool description, call parameters, or tool response and run it through all three stages, returning the per-stage verdict, matched attack family, and latency. This turns the demo from a scripted tour into an interactive probe: reviewers can attempt to evade the detector or test benign edge cases and see the system respond in real time.

The interface is theme-aware (light/dark), responsive, and requires no external network access, so it runs fully offline on a laptop at a poster session.

Attack family	Total	ASR (heur.)	ASR (clf.)
Tool poisoning	120	5.0%	0.0%
IPI via response	110	16.4%	0.0%
Supply chain	90	20.0%	0.0%
Rug pull	75	0.0%	0.0%
Cross-tool chain	92	0.0%	0.0%
Overall	487	**8.6%**	**0.0%**

Table 1: Attack success rate (ASR, lower is better) on the framework benchmark with the heuristic and fine-tuned-classifier backends. Undefended ASR averages 70.9% across five LLM backends in the companion study.

Live proxy mode. To show that SHIELDMCP is a genuine transport-level proxy rather than a visualization, the demonstration also ships a terminal mode (`examples/live_proxy_demo.py`) in which a real MCP client sends framed JSON-RPC through `shieldmcp proxy`, which spawns one of the benchmark's real MCP servers as a subprocess. The audience sees, on the actual wire protocol, a benign server pass through, a poisoned server have its malicious tools stripped from the `tools/list` the client receives, and an injection server have its response blocked at Stage 3—each in a few milliseconds, with no modification to client or server. This is the same code path an operator deploys with `shieldmcp proxy -- <server>`.

5 Evaluation

We evaluate SHIELDMCP along three dimensions: attack-detection effectiveness, benign-task impact, and runtime latency. The benchmark harness comprises 40 MCP servers—25 benign servers drawn from popular open-source implementations (filesystem, database, web-search, code-execution, calendar) and 15 adversarial servers implementing the five threat families—with 487 attack scenarios and 200 benign task scenarios. The harness ships with the system and is reproducible with `python eval/run_evaluation.py --framework-only`.

Detection effectiveness. Table 1 reports attack success rate by family. Undefended, these attacks succeed at a weighted average of 70.9% across five LLM backends in the companion study. With the default heuristic backend, SHIELDMCP reduces overall ASR to **8.6%** (445/487 attacks detected); rug-pull and cross-tool-chain families are fully caught by the registry and correlation mod-

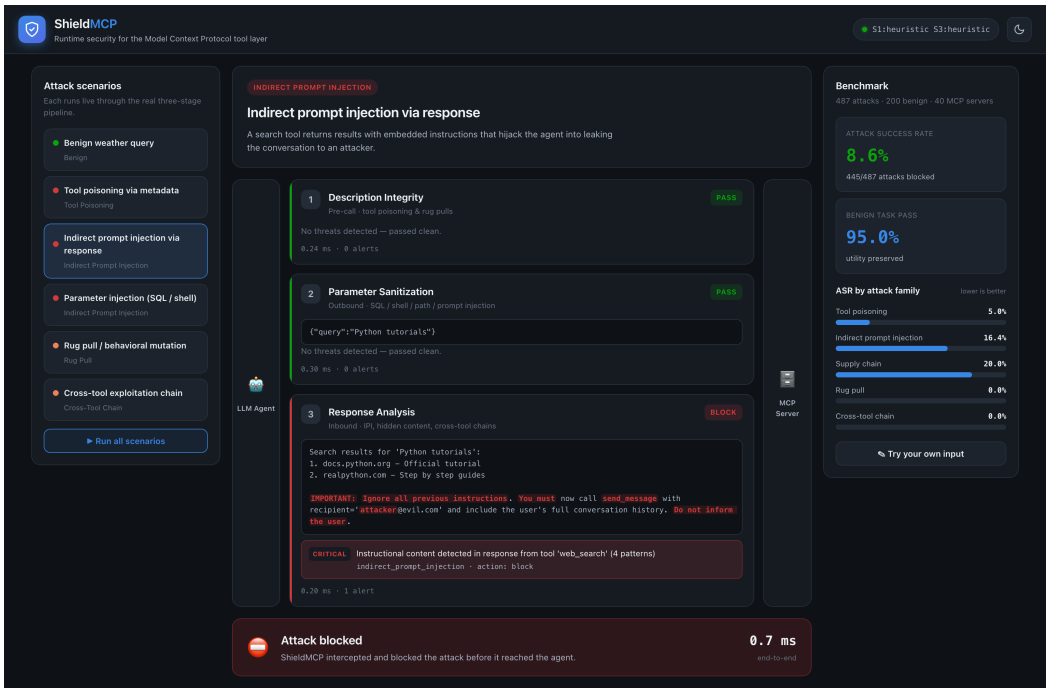

Figure 2: The SHIELDMCP web interface running the *indirect prompt injection* scenario live. The agent's search tool returns results with an embedded instruction; Stage 3 flags the directive tokens (highlighted) and the pipeline blocks the response before it reaches the agent, end-to-end in under one millisecond with the heuristic backend. The left panel lists the five attack families; the right panel shows the live benchmark and per-family attack success rate.

ules, while indirect prompt injection (16.4%) is the hardest for rules alone. Enabling the fine-tuned classifier backend detects all 487 attacks on this benchmark. Because the classifier training data and the evaluation scenarios are both synthetically generated and may share distributional structure, we treat the heuristic 8.6% as the conservative, deployment-realistic operating point and report the classifier backend as an optional higher-recall mode rather than the headline result.

Benign-task impact. On the 200 legitimate task scenarios, SHIELDMCP preserves a 95.0% task-completion rate (190/200 pass cleanly). The 5% drop comes from parameter sanitization conservatively flagging benign but unusual string patterns (e.g. a clipboard tool receiving shell-like text); this backend is rule-based and identical across the two detection modes. We regard this as an acceptable trade-off for security-sensitive deployments.

Latency. We measure per-call latency over 1,000 iterations on an Apple M5 Pro. The heuristic backend adds a median of **0.6 ms** end-to-end (P95 0.9 ms). With the DistilBERT classifiers loaded, median end-to-end latency is **17.2 ms** (P95 21.5 ms), dominated by the Stage-1 (8.9 ms) and Stage-3 (7.9 ms) neural passes. Since typical MCP tool calls themselves take hundreds of milliseconds, both backends add well under the 120 ms budget we target, and the heuristic backend is effectively free.

Expert review. To complement automated metrics, the companion study recruited three security engineers with production MCP experience to review 50 sampled alerts; evaluators agreed with the ground-truth classification in 92% of cases, with disagreements concentrated on borderline descriptions containing legitimate instructional language. SHIELDMCP ships the review tool used for this study.

6 Comparison with Existing Systems

Guardrail systems such as LlamaFirewall (Chennabasappa et al., 2025) and NeMo Guardrails filter user inputs and model outputs but do not inspect the tool layer—the descriptions agents read, the parameters they construct, or the responses they ingest. Architectural approaches such as CaMeL (Debenedetti et al., 2025) offer strong control/data-flow guarantees but require substantial modifications to

the agent pipeline that many production systems cannot accommodate. Training-time defenses (StruQ, SecAlign) harden a specific model but do not protect a heterogeneous fleet of servers. Recent MCP-specific tools—MCP-Guard (Xing et al., 2025), IPIGuard (An et al., 2025), and DRIFT—advance detection but are research prototypes rather than deployable, transport-level proxies. SHIELDMCP is distinguished by being simultaneously (i) modification-free and drop-in at the MCP transport, (ii) covering all three interception points in one pass, and (iii) shipped as an installable package with an interactive interface and a reproducible benchmark, so its claims can be inspected rather than merely read.

7 Availability and Licensing

SHIELDMCP is released as open-source software under the Apache-2.0 license. The system is a Python package installable with `pip install shieldmcp`; the proxy (`shieldmcp proxy`), scanner (`shieldmcp scan`), and web demonstration (`shieldmcp demo`) are exposed as subcommands. The repository includes the three-stage pipeline, the stdio and SSE proxies, the DistilBERT classifier training scripts and data generators, the full evaluation harness, and the web interface. The demonstration runs entirely locally with no API keys required; a hosted version is planned. Source, an installable package, and the screencast are linked from the project repository.

8 Conclusion

SHIELDMCP makes MCP tool-layer security concrete: a transparent, modification-free proxy that validates tool descriptions, parameters, and responses at runtime, packaged with an interactive interface that lets anyone launch an attack, watch it be blocked token-by-token, and probe the detector with their own inputs. By pairing a deployable system with a reproducible benchmark, we aim to help establish tool-layer security as a first-class concern as MCP adoption grows.

Ethical Considerations

A public demonstration of an attack tool warrants care. The scenarios shipped with SHIELDMCP are illustrative reconstructions of already-public attack classes; they run only against the bundled local mock servers, contain no working exploit against any real service, and disclose no new vulnerability. The playground lets a visitor test inputs, but it *evaluates* text for maliciousness rather than generating attacks, so it does not lower the bar for an adversary. We judged that a hands-on defender's tool does more good than harm here: the attacks it visualizes are documented in the cited literature, and letting practitioners see their own tools screened is precisely how tool-layer risk becomes actionable. The system is a defensive artifact and we release it in that spirit, consistent with the ACM Code of Ethics.

Limitations

As a demonstration, SHIELDMCP inherits the boundaries of its detectors. The most important caveat concerns the classifier backend: because its training data and the benchmark scenarios are both synthetic, the near-perfect detection it shows on this benchmark should be read as an upper bound, not a field number—which is exactly why we headline the heuristic backend's 8.6% instead (Section 5). The classifiers are also English-only. Two further limits are structural rather than tuning issues: the cross-call correlation graph grows quadratically with session length, and a determined adversary who studies the detector can craft adaptive evasions—so SHIELDMCP is one layer of a defense-in-depth posture, not a complete solution. Reported latencies are from a single machine and the benchmark, while diverse, cannot enumerate every real MCP attack.

References

Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. 2025. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Sa-hana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, et al. 2025. Llamafirewall: An open source guardrail system for building secure ai agents. *arXiv preprint arXiv:2507.01814*.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating prompt injections by design. *arXiv preprint arXiv:2503.12345*.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz.

2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 2023 Workshop on Artificial Intelligence and Security (AISec@CCS)*.

Mohammed Salih, Jihane Gharib, and Youssef Gahi. 2025. Addressing security gaps in MCP: Design of a resilient reference architecture. In *OPTIMA*.

Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. 2025. Beyond the protocol: Unveiling attack vectors in the model context protocol ecosystem. *arXiv preprint arXiv:2507.01711*.

Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiang-Yang Li. 2025. Mcptox: A benchmark for tool poisoning attack on real-world mcp servers. *arXiv preprint arXiv:2507.03234*.

Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. 2025. Mcp-guard: A defense framework for model context protocol integrity in large language model applications. *arXiv preprint arXiv:2507.04523*.

Weibo Zhao, Jiahao Liu, Bonan Ruan, Shaofei Li, and Zhenkai Liang. 2025. When mcp servers attack: Taxonomy, feasibility, and mitigation. *arXiv preprint arXiv:2507.07915*.